Practical No 04

Aim: Write a program to perform merge sort on the given two list of integer values.

Theory:

Merge Sort

Merge Sort is a **divide and conquer** algorithm that recursively divides the list into two halves, sorts them, and then merges them back into a sorted sequence. It has a **time complexity of O(n log n)** in all cases, making it efficient for large datasets. The merging step ensures that elements are combined in a sorted order.

Key Steps in Merge Sort:

1. Divide: Split the list into two halves.
2. Conquer: Recursively sort both halves.
3. Merge: Combine the sorted halves into a single sorted list.

Example of Merge Sort

Let's take an example of sorting the array:

Input: [8, 3, 5, 2, 7, 6, 4, 1]

Step 1: Divide the array

Split it into two halves:
[8, 3, 5, 2] and [7, 6, 4, 1]

Further divide each subarray:
[8, 3] and [5, 2]
[7, 6] and [4, 1]

Keep splitting until each subarray has a single element:
[8] [3] [5] [2] [7] [6] [4] [1]

Step 2: Conquer (Sort and Merge the subarrays)

Merge [8] and [3] → [3, 8]
Merge [5] and [2] → [2, 5]
Merge [7] and [6] → [6, 7]
Merge [4] and [1] → [1, 4]

Now, merge these sorted subarrays:
Merge [3, 8] and [2, 5] → [2, 3, 5, 8]
Merge [6, 7] and [1, 4] → [1, 4, 6, 7]

Step 3: Final Merge

Merge [2, 3, 5, 8] and [1, 4, 6, 7] → [1, 2, 3, 4, 5, 6, 7, 8]

Final Sorted Array

[1, 2, 3, 4, 5, 6, 7, 8]

This is the **sorted version** of the original list.

Program:

```
# Function to perform Merge Sort
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2  # Finding the middle index
        left_half = arr[:mid]  # Dividing the array into left half
        right_half = arr[mid:]  # Dividing the array into right half

        merge_sort(left_half)  # Recursively sorting the left half
        merge_sort(right_half)  # Recursively sorting the right half

        i = j = k = 0  # Pointers for left_half, right_half, and merged array

        # Merging the two halves in a sorted manner
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]  # Adding smaller element from left_half
                i += 1
            else:
                arr[k] = right_half[j]  # Adding smaller element from right_half
                j += 1
            k += 1  # Move to the next position in the merged array
```

```
    # Copy any remaining elements from left_half (if any)
    while i < len(left_half):
        arr[k] = left_half[i]
        i += 1
        k += 1

    # Copy any remaining elements from right_half (if any)
    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1

# Function to merge two lists and sort them using Merge Sort
def merge_and_sort_lists(list1, list2):
    merged_list = list1 + list2  # Merging both lists into one
    merge_sort(merged_list)  # Sorting the merged list using Merge Sort
    return merged_list  # Returning the sorted merged list

# Taking user input for the first list
list1 = list(map(int, input("Enter elements of first list separated by spaces: ").split()))
# Taking user input for the second list
list2 = list(map(int, input("Enter elements of second list separated by spaces: ").split()))

# Calling function to merge and sort the lists
sorted_list = merge_and_sort_lists(list1, list2)

# Printing the original and sorted lists
print("List 1 is:", list1)
print("List 2 is:", list2)
print("Sorted Merged List:", sorted_list)
```

Output:

4.Merge_sort.py - D:\Programming\LABS SEM 4\DAA\4.Merge_sort.py (3.12.4)

File Edit Format Run Options Window Help

```python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2  # Finding the middle of the array
        left_half = arr[:mid]  # Dividing the elements into 2 halves
        right_half = arr[mid:]

        merge_sort(left_half)  # Sorting the first half
        merge_sort(right_half)  # Sorting the second half

        i = j = k = 0

        # Copy data to temp arrays left_half and right_half
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Checking if any element was left
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1

def merge_and_sort_lists(list1, list2):
    merged_list = list1 + list2  # Merging two lists
    merge_sort(merged_list)  # Sorting using merge sort
    return merged_list

# Taking user input for two lists
list1 = list(map(int, input("Enter elements of first list separated by spaces: ").split()))
list2 = list(map(int, input("Enter elements of second list separated by spaces: ").split()))

sorted_list = merge_and_sort_lists(list1, list2)
print("List 1 is:", list1)
print("List 2 is:", list2)
print("Sorted Merged List:", sorted_list)
```

Ln: 20 Col: 18

IDLE Shell 3.12.4

File Edit Shell Debug Options Window Help

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun  6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:\Programming\LABS SEM 4\DAA\4.Merge_sort.py
Enter elements of first list separated by spaces: 1 9 5 7 3
Enter elements of second list separated by spaces: 8 4 6 2
List 1 is: [1, 9, 5, 7, 3]
List 2 is: [8, 4, 6, 2]
Sorted Merged List: [1, 2, 3, 4, 5, 6, 7, 8, 9]
>>>
```

Ln: 10 Col: 0

Conclusion:

Hence I have successfully performed a perform of merge sort on the two list of integer values.